Ch 03: Logical Operators and Boolean Variables

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

Logical Operators

- **Logical operators** operators that can be used to create complex Boolean expressions
  - `and` operator `or` operator
    - connect two Boolean expressions into a compound Boolean expression
  - `not` operator
    - unary operator, reverses the truth of its Boolean operand

Logical Operators Concept

The logical `and` operator and the logical `or` operator allow you to connect multiple Boolean expressions to create a compound expression. The logical `not` operator reverses the truth of a Boolean expression.

EAST TEXAS A&M

Logical Operator Truth Tables

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

A	not A
False	True
True	False

Short-Circuit Evaluation

- **Short circuit evaluation**: deciding the value of a compound Boolean expression after evaluating only one sub expression
  - Performed by the `or` and `and` operators
  - Left to right order of evaluation
    - For `or` operator: as soon as operand from left to right evaluates as `True` you can stop, you know expression is `True`
    - For `and` operator: as soon as operand from left to right evaluates as `False` you can stop, you know expression is `False`

Compound Boolean Expressions using Logical Operators

Table 4: Compound Boolean expressions using logical operators

Expression	Meaning
`x > y and a < b`	Is `x` greater than `y` AND is `a` less than `b`?
`x == y or x == z`	Is `x` equal to `y` OR is `x` equal to `z`?
`not (x > y)`	Is the expression `x > y` NOT true?
`x >= 10 and x <= 20`	Is `x` in the range 10 to 20 (inclusive)
`x < 10 or x > 20`	Is `x` not in the range 10 to 20 (inclusive)

Checking Numeric Ranges with Logical Operators

- Determine if a value is within a specific range
  - `x >= 10 and x <= 20`
- Determine whether a numeric value is outside of a specific range
  - `x < 10 or x > 20`

Note comparison operators have higher precedence, so don't need to put parentheses in these expressions (though will not hurt)

Note short-circuiting, for example if `x = 5`, and we are performing the first test to see if `x` is within the range, since `5 >= 10` is `False` the evaluation of this expression stops because we know the `and` will be `False` no matter what the right hand expression is.

Boolean Variables

Boolean Variables Concept

A Boolean variable can contain one of two values: `True` or `False`. Boolean variables are commonly used as flags, which indicate whether specific conditions exist or not.

- **Boolean variable**: contains one of two values, `True` or `False`
  - Represented by `bool` data type
- Commonly used as flags
  - **Flag**: variable that signals when some condition exists in a program
    - Flag set to `False` → condition does not exist
    - Flag set to `True` → condition exists

```
>>> sales = 25000
>>> sales_quota_met = sales >= 50000.0
>>> sales_quota_met
False
>>> type(sales_quota_met)
<class 'bool'>
```

```
>>> sales = 75000
>>> sales_quota_met = sales >= 50000.0
>>> sales_quota_met
True
>>> type(sales_quota_met)
<class 'bool'>
```

EAST TEXAS A&M

Summary

- Logical operators `and` `or` and `not` allow you to connect multiple Boolean expressions to create compound expressions
  - This is useful to simplify and remove nesting of conditions
- A common use of Boolean Expressions is to check values are within or outside of needed range
- Python has a basic `bool` data type, it holds a single Boolean value, either `True` or `False`
  - The `bool` Boolean variable is useful as a **Flag** in a program

EAST TEXAS A&M

References